

PyTorch 2.0 TTNN: Tenstorrent Integration

Overview

Introduction to TTNN and Tenstorrent Integration

The **PyTorch 2.0 TT-NN Compiler (TTNN)** is an extension of PyTorch that enables models to run on Tenstorrent's AI accelerators ¹. It maintains the familiar PyTorch API while offloading computations to Tenstorrent hardware for improved performance ¹. In practice, TTNN introduces a new device type (for Tenstorrent cards) and a backend compiler so that PyTorch operations can execute on Tenstorrent hardware either in **eager mode** or via **graph compilation** ² ³. This allows an experienced PyTorch developer to treat a Tenstorrent accelerator similarly to CUDA or CPU: by moving tensors/models to the **TT device**, computations will occur on the accelerator using TTNN's implementation of operations. The goal is to achieve significant speedups on Tenstorrent's RISC-V based many-core architecture, all while using standard PyTorch model code ¹ ⁴.

Tenstorrent Hardware & TTNN Library: Tenstorrent's accelerators (like Grayskull, Wormhole) consist of a grid of Tensix cores (each a cluster of small RISC-V CPUs with attached tensor units) connected by a network-on-chip ⁵ ⁶. Unlike GPUs, they lack unified global memory or caches – instead each core has local SRAM, and data is moved explicitly via DMA, taking advantage of the physical core layout for efficiency ⁷ ⁸. To utilize this hardware, Tenstorrent provides two software stacks: a high-level compiler (formerly “tt-buda”, being supplanted by an MLIR-based flow) and a low-level API called **TT-Metal** ⁹. TT-Metal is analogous to a compute runtime (similar to ROCm or OpenCL in spirit) giving fine-grained control over kernels and memory. **TTNN (Tenstorrent Neural Network library)** is built on TT-Metal as a higher-level tensor-and-operator library, providing a NumPy/PyTorch-like interface for tensor operations ¹⁰. TTNN handles details like **tiled tensor layouts** (breaking tensors into hardware-friendly tiles with padding) and supports Tenstorrent-specific data types (e.g. BFloat8, BFloat16) that the hardware uses for efficiency ¹¹ ¹². In short, TTNN serves as the bridge between PyTorch and the Tenstorrent hardware, managing device memory, data format conversions, and kernel launches on the accelerator.

Eager Mode Execution in TTNN

Eager mode means PyTorch operations execute immediately on the Tenstorrent device, operation by operation, just as they would on CPU or CUDA in PyTorch's default eager execution. With TTNN, using eager mode is as simple as moving your model or tensors to the Tenstorrent device. For example:

```
import torch
import torch_ttnn # PyTorch TTNN extension
model = YourModel()

# Open Tenstorrent device and get a torch.Device for it
device = ttnn.open_device(device_id=0)
```

```

tt_device = torch_ttnn.ttnn_device_as_torch_device(device) # wrap TT device as
torch.device

model.to(tt_device) # Move model parameters to Tenstorrent device
output = model(input_data)
# Forward pass executes on Tenstorrent hardware in eager mode

```

In this mode, each tensor operation (matrix multiply, convolution, ReLU, etc.) on tensors residing on the TT device is dispatched through PyTorch's dispatcher to a TTNN-provided kernel implementation ² ¹³. Internally, PyTorch recognizes the tensor's device type as Tenstorrent (TT) and routes the operation to TTNN's C++ kernel instead of the default CPU/CUDA kernel. (Under the hood, TTNN registers its device and kernels using PyTorch's **PrivateUse1** backend interface – a mechanism for out-of-tree backends. This allows defining a custom device type without modifying PyTorch's core source ¹⁴ ¹⁵. TTNN uses this to map Tenstorrent as a new device type at runtime, so that `torch.device("tt", 0)` or similar becomes valid, backed by PrivateUse1 dispatch keys.)

How Eager Dispatch Works: When an op is called on TTNN tensors, PyTorch's ATen dispatcher will invoke the TTNN kernel registered for that op (via `TORCH_LIBRARY_IMPL` for the PrivateUse1 backend) ¹⁵. For example, an addition `x + y` (if `x, y` are on the TT device) triggers the TTNN implementation of `aten::add.Tensor`. That TTNN kernel is a C++ function (in the `torch_ttnn` extension) which likely performs the following steps:

1. **Input handling:** It receives the input tensor handles (which wrap TTNN's internal Tensor objects). If the inputs are not already in Tenstorrent's native layout or on the device, it will convert or move them as needed. (Typically, tensors are moved to the device beforehand as shown with `model.to(tt_device)`, but operations like implicit type conversion or scalar expansion might need handling.)
2. **Call TTNN C++ API:** The kernel calls into the TTNN library's corresponding operation, e.g. `ttnn::add()` in C++. The TTNN library (built on TT-Metal) then creates a device **program** for the operation. This may involve tiling the data (if not already tiled) and issuing low-level TT-Metal commands to perform the add on the Tenstorrent hardware ¹⁶ ¹⁷. For an elementwise op like add, the library might simply enqueue a vector addition kernel on each relevant Tensix core.
3. **Device execution:** The TTNN library uses TT-Metal to launch the kernel on the accelerator. The actual computation happens on the Tenstorrent card. The result resides in the device's memory (possibly in tiled format).
4. **Return result:** TTNN returns a handle to the resulting tensor. The PyTorch TTNN extension wraps this into an `at::Tensor` with device type TT. From the PyTorch user's perspective, the result is again a `torch.Tensor` on the Tenstorrent device.

This flow is illustrated below for a single operation in eager mode:

```

flowchart LR
    UserCodeE[User PyTorch Code<br/>(e.g., `y = x + z`)] --> DispatcherE[PyTorch<br/>Dispatcher<br/>(routes op to TT backend)]
    DispatcherE --> KernelE[TTNN Eager Kernel<br/>(C++ op implementation)]
    KernelE --> TTNNLibE[TTNN C++ Library<br/>(calls TT-Metal API)]

```

```
TTNNLibE --> MetalE[TT-Metal Driver & Firmware]
MetalE --> HardwareE[Tenstorrent Accelerator Hardware]
```

In this diagram, **PyTorch's dispatcher** detects that `x` and `z` are on a Tenstorrent device and thus dispatches the `+` operation to the TTNN kernel. The TTNN C++ kernel in turn calls into **TTNN library** functions which prepare and execute the operation on the **hardware** via TT-Metal. The result then propagates back up to PyTorch. All of this happens synchronously in eager mode (from the user's perspective): by the time the `+` operation returns, the computation has completed on the device, and `y` is ready for use (residing on the TT device).

Memory format and transfers: One important aspect is that Tenstorrent uses a **tiled data layout** internally for performance (e.g., 32×32 tiles) which differs from PyTorch's row-major layout ¹¹ ¹². TTNN handles this transparently. When you call `model.to(tt_device)`, under the hood TTNN likely converts (**tilizes**) the tensors to the tiled layout and copies them to the device memory. Likewise, if you fetch a result back to CPU (via `tensor.cpu()` or `ttnn.to_torch()`), TTNN will **untilize** (reassemble) the tiles into a contiguous torch tensor on host memory. For example, TTNN provides helper APIs like `ttnn.from_torch()` and `ttnn.to_torch()` to convert between torch Tensors and TTNN's format ¹⁸ ¹⁹. A snippet from the TTNN docs shows converting a torch tensor to TTNN (with tiling) and back:

```
torch_input = torch.zeros(2, 4, dtype=torch.float32)
tt_tensor = ttnn.from_torch(torch_input, dtype=ttnn.bfloat16,
                             layout=ttnn.TILE_LAYOUT)
torch_output = ttnn.to_torch(tt_tensor)
```

Here, `from_torch` copies and tiles the data (and converts dtype to bfloat16 as required by the device) and `to_torch` brings the result back to CPU ¹⁸ ¹⁹. This makes debugging easier – you can inspect TTNN tensors by converting them to regular torch tensors when needed.

Autograd and Backward: In eager mode, PyTorch's autograd should work largely as usual for TTNN operations, but under the covers it uses the **AutogradPrivateUse1** key to handle gradients. If TTNN provides backward implementations for certain ops, those will be used; otherwise PyTorch might fall back to using the operation's mathematical definition to compute the gradient (possibly on CPU). The exact details depend on TTNN's implementation. In many cases, if an operation's forward is on the TT device but no backward is implemented, PyTorch can perform backward by doing the backward pass on CPU (this would involve pulling needed data back to CPU, which is slow). For initial development, TTNN might rely on this kind of fallback for autograd. One can also implement custom backward functions in the TTNN library for better performance. As an eager-mode developer, keep in mind that supporting training means implementing both forward and backward or ensuring the autograd engine can safely fallback.

Fallback for Unsupported Ops: A key feature for an evolving backend is the ability to **fallback** to default PyTorch (CPU) implementations when an op is not yet supported on the Tenstorrent device. TTNN uses PyTorch's boxed fallback mechanism to achieve this. In fact, PyTorch's dispatcher allows registering a **backend fallback** kernel that catches any op not otherwise implemented for PrivateUse1 and handles it ²⁰. TTNN likely registers a fallback that does the following: transfer the tensors to CPU, perform the operation with PyTorch's CPU kernel, and then bring the result back to the TT device. This way, **models can**

run end-to-end even if some ops aren't implemented on the accelerator, at the cost of performance. The TTNN README's model status table indicates when a model runs fully on device ("End-to-end on device: All PyTorch operations have been converted to TT-NN operations" ²¹) versus when some ops fell back ("Compiled: ... some operations still fallback to PyTorch" ²²). For example, if an `einsum` or a non-core operation appears in your model and TTNN doesn't support it, the backend will use the CPU op and incur a device transfer (these show up as `to/from_device` ops in their stats).

From a developer perspective, you can also manually invoke fallback if needed. The TTNN API provides `ttnn.get_fallback_function(op)` which returns a Python function wrapping the torch operator for fallback use. The documentation gives an example of two ways to apply an unsupported activation (SiLU) in a TTNN workflow ²³ :

- **Recommended:** Convert the TTNN tensor to a torch tensor, use the torch operation, then convert back:

```
python
tensor = ttnn.to_torch(tt_tensor)          # to CPU
tensor = torch.nn.functional.silu(tensor)   # use CPU op
result = ttnn.from_torch(tensor, dtype=ttnn.bfloat16) # back to TTNN tensor
```

²³

- **Alternative:** Use `ttnn.get_fallback_function`:

```
result = ttnn.get_fallback_function(ttnn.silu)(tt_tensor)
```

This will internally do the same steps for certain ops (though the docs note this only works for *some* operations) ²⁴ .

Using these fallback tools, you can debug or bypass unsupported parts of models. When developing new op support, it's common to first rely on the fallback to ensure correctness, then incrementally replace it with a native implementation.

Local Debugging Without Hardware: If you don't have Tenstorrent hardware on hand, you can still start developing and testing many aspects of eager mode. One approach is to use the CPU fallback path intentionally for all ops, essentially using PyTorch CPU as a "simulation" of the TT device. For instance, you could open a TT device in a mode that doesn't actually require the card (if supported), or simply ensure that your operations call the fallback. PyTorch's testing of custom backends often simulates a device by storing data in CPU memory under the hood ²⁵ . In fact, the PyTorch `open_registration_extension.cpp` test demonstrates a dummy backend where the custom device's data is just on CPU and a fallback does the heavy lifting ²⁵ . TTNN doesn't advertise a full CPU simulation mode, but you can approximate one by using the fallback functions to execute ops. For example, to test your new op implementation, you might initially route it to a golden reference (torch CPU op) and verify numerics before hooking up the real device code. Also, you can write unit tests that compare `ttnn` results to pure PyTorch easily by converting tensors back and forth (as shown above). Logging and tracing can help as well - TTNN allows enabling logs (e.g., `ttnn.enable_logging()` in their examples) to see what ops are executed, and even capturing a graph of operations for analysis ²⁶ ²⁷ . In summary, while you obviously cannot achieve speedups without the

actual hardware, you can ensure your integration code is functionally correct by leveraging PyTorch's robust CPU ops as a reference. Then, when hardware is available or via remote access (Tenstorrent provides options like Koyeb cloud instances for their cards ²⁸), you can test on the real accelerator to measure performance and catch hardware-specific issues.

Compiled Mode Execution (Graph Mode)

While eager mode prioritizes ease of use and rapid prototyping, **compiled mode** is where TTNN unleashes the full performance potential of Tenstorrent hardware. In PyTorch 2.0, compiled mode is accessed via `torch.compile`, which uses TorchDynamo and TorchInductor (or a backend of your choice) to trace and optimize your model ahead-of-time. TTNN provides a custom backend for `torch.compile`, invoked by passing `backend=torch_ttnn.backend` and an options object to `torch.compile` ³. For example:

```
import torch
import torch_ttnn
model = YourModel()

device = ttnn.open_mesh_device(ttnn.MeshShape(1, 2))
# e.g., using a 1x2 device mesh (multi-chip)
options = torch_ttnn.TorchTtnnOption(device=device, data_parallel=2)
model = torch.compile(model, backend=torch_ttnn.backend, options=options)
output = model(input_data) # This will run the compiled graph on Tenstorrent hardware
```

In this mode, the first invocation of the model will trigger a tracing/compilation process: TorchDynamo intercepts the model execution to produce an FX graph (a high-level graph of PyTorch operations), which is then lowered to TTNN for code generation. The **TTNN backend** compiles the entire graph (or large parts of it) into an optimized program for the Tenstorrent device. Subsequent model invocations reuse the compiled binary, leading to much faster execution after the one-time compile cost ²⁹. The TTNN README's performance table shows dramatic speedups here – for example, ResNet18's throughput improved from ~0.42 to ~73.2 inferences/sec using TTNN (that original likely on CPU) ⁴, and large models like BERT jumped from essentially 0 (no throughput on CPU) to ~40 inferences/sec ³⁰ ³¹, indicating the Tenstorrent accelerator (with TTNN's compiled execution) vastly outperforms the baseline.

The compiled execution flow can be visualized as follows:

```
flowchart LR
    UserGraph[User Model (via `torch.compile`)] --> Dynamo[TorchDynamo Trace (FX Graph)]
    Dynamo --> CompileBackend[TTNN Backend (Graph Lowering & Compile)]
    CompileBackend --> TTNNLibGraph[TTNN Graph Executor<br/>(C++ compiled run)]
    TTNNLibGraph --> MetalGraph[TT-Metal Driver & Runtime]
    MetalGraph --> HardwareGraph[Tenstorrent Hardware]
```

Trace and Lowering: In the above flow, when you call `torch.compile(model, backend=torch_ttnn.backend)`, **TorchDynamo** suspends execution and records the operations as a graph. That graph is then passed to TTNN's compile backend. TTNN's backend likely uses components of **TorchInductor** or a similar mechanism to lower the FX graph into device-specific code. This could involve several steps:

- **Operator Conversion:** Each PyTorch op in the graph is either converted to an equivalent TTNN operation or a sequence of TTNN operations. Since TTNN has its own library of operations (many of which mirror PyTorch/ATen ops), the compiler will map the FX nodes to TTNN library calls. If certain high-level PyTorch ops have no direct TTNN equivalent, the compiler may decompose them. (It's also possible that TTNN's backend relies on Inductor's ability to break things down to core ATen ops, many of which TTNN supports.)
- **Graph Optimizations:** With the whole graph visible, TTNN can perform optimizations that are impossible in eager mode. For example, it can **fuse operations** to reduce intermediate writes (especially important given the small SRAM on each core) and to avoid extra round trips to host. It can also **optimize data layout and transfers** – e.g., ensure that intermediate results remain in on-chip memory as long as possible, or overlap communications. Tenstorrent's design encourages minimizing DRAM accesses and keeping data local to cores, which a compiled approach can plan for. The TTNN documentation suggests that without graph-level scheduling, one cannot fully utilize SRAM across multiple ops ³² ³³. The compiled mode addresses this by analyzing the entire model graph.
- **Kernel Generation:** The backend might generate native Tenstorrent device code (possibly via TT-Metal's API or an MLIR codegen). It could create a sequence of TT-Metal calls corresponding to the graph, or even a fused kernel that implements multiple operations in one program. (Tenstorrent announced an MLIR-based flow, so TTNN might eventually integrate with that, but currently it likely uses its C++ API and perhaps templated kernel libraries.)
- **Launching the Graph:** The end result is that TTNN prepares an executable that encompasses the model's forward (and possibly fixed batch size, etc., as compiled). When you run the model, this binary is loaded to the card and executed, producing the output. The first run includes compile time (which is why the README shows a separate metric for "Compiled First Run (ms)" including one-time overhead ²⁹). Later runs skip compilation and go straight to execution, thus achieving high throughput.

From a user perspective, compiled mode is mostly transparent – you use `torch.compile` and then the model call just works. Under the hood, however, it's doing heavy lifting to maximize performance. For instance, **multi-device support** is handled here: the example above uses `open_mesh_device(ttnn.MeshShape(1,2))` and `data_parallel=2` in the options ³⁴. This suggests TTNN can partition the model across a mesh of devices or run data parallel replication. The compiled backend would take care of splitting the graph or inputs accordingly and orchestrating execution on multiple Tenstorrent chips in parallel.

Partial Graph Fallbacks: If the compiler encounters an op or pattern it cannot lower to the TT device, it might split the graph and let unsupported parts run on CPU (or GPU) as a fallback. In such cases, TorchDynamo will insert calls to those ops on the host, and data will transfer to CPU and back to device. This is reflected in the model support table: "Torch Ops Remain" indicates ops that were not converted ³⁵. For example, a model might show a few remaining torch ops – those ran in the fallback path, incurring some `to_device` / `from_device` overhead ³⁵. The TTNN team treats those models as "Compiled (with

fallbacks)" in the table, vs. fully end-to-end ones. As you develop new op support, one big payoff is moving models from the "some fallbacks" category to "fully on device" (), which eliminates those costly transfers.

In summary, compiled mode uses ahead-of-time optimization to bundle many operations into one efficient execution unit on Tenstorrent. It is **recommended for performance-critical scenarios** ³, whereas eager mode is great for quick iteration, debugging, and when first bringing up model support.

Code Structure and Key Components of the TTNN Project

Understanding how TTNN is implemented will guide you in extending its functionality. The project is structured into several parts ³⁶:

- `torch_ttnn/` – the core Python package and C++ extension of TTNN. This contains the integration with PyTorch (device registration, operator implementations, etc.), as well as Python APIs exposed to users (like `ttnn.open_device`, `ttnn.from_torch`, etc.).
- `tracer/` – likely contains graph tracing or conversion utilities, possibly used by the compile backend. (This could be custom code to assist TorchDynamo or to handle graph capture for debugging.)
- `docs/` – documentation and design reports (for example, files outlining how operations are lowered, how the device integration works, etc., which are valuable to read as a developer).
- `tests/` – a suite of tests, including unit tests for ops and end-to-end tests on various models (the same models listed in the support table). These use `pytest` ³⁶. Studying tests can help understand expected behaviors. (Keep in mind some tests are likely marked to only run if hardware is present, to validate real execution.)
- `demo/` – example scripts showing how to use TTNN in practice (similar to those in documentation, demonstrating both eager and compile modes on toy models).
- Additional files like `setup.py`, build configs, etc., to compile the extension.

Focusing on the **C++ integration**: When you open a Tenstorrent device via `ttnn.open_device(0)`, it likely calls into a C++ function that uses TT-Metal to initialize the device (e.g., open a context to the card with ID 0). This returns a handle (perhaps a pointer or an id) to a `Device` object. The `torch_ttnn.ttnn_device_as_torch_device(device)` then wraps that into a `torch.device`. Internally, PyTorch identifies custom devices by an index and a type. TTNN probably registers itself under one of the reserved **PrivateUse** device types. For instance, PyTorch reserves `DeviceType::PrivateUse1` (and 2,3) for out-of-tree backends ³⁷. TTNN might use `PrivateUse1` and then **rename** it to something like `"tt"` or `"tenstorrent"` so that `torch.device("tt", 0)` becomes valid. (The PyTorch developer docs mention that you can alias the `PrivateUse1` device to a name for user convenience ³⁸ – e.g., Huawei's Ascend NPU backend uses `PrivateUse1` but appears as `npu` device in PyTorch.)

During initialization, TTNN's extension would register several things with PyTorch:

- A **DeviceType** with its device index range (so PyTorch knows about `device=TT:0`, etc.).
- A **DeviceGuard** implementation: this is for PyTorch's device context handling. (E.g., when you do `with torch.device("tt:0"):` or when PyTorch needs to set the current device, it uses this to push/pop device state, similar to `cudaSetDevice` in CUDA.)

- **A Allocator/Storage** implementation: to manage memory for TTNN tensors. This is critical – when PyTorch needs to allocate a tensor on the TT device, it will call TTNN's allocator. TTNN's allocator likely allocates memory via TT-Metal (which might be pinned host memory or device DRAM). It could also allocate a proxy object since actual memory management might be opaque under TT-Metal. Notably, **TTNN tensors might not have a typical “Storage” like CPU/CUDA**; they could just hold a reference to a TT-Metal buffer. But PyTorch needs to be aware of sizes and have a way to free that memory, hence a custom storage/allocator is registered.
- **Kernels for operations:** Using `TORCH_LIBRARY_IMPL(aten, PrivateUse1, m)`, TTNN registers implementations for many operations. This is essentially a giant table of function pointers for things like `add`, `matmul`, `conv2d`, etc. Each implementation is a C++ function in the extension that typically unpacks the `Tensor` arguments (getting the underlying TTNN tensor references) and calls the appropriate TTNN library routine. For example, there might be a function `Tensor ttnn_add(const Tensor& a, const Tensor& b)` that calls `ttnn::add(*a.tt_tensor, *b.tt_tensor)` internally, then wraps the result in a new PyTorch tensor. The PyTorch tutorial on extending with PrivateUse1 shows a simple example of registering an `add` kernel in this way ¹⁵. TTNN would do this for dozens of operators. If an operator is not registered, the dispatcher will use the fallback as discussed. (TTNN's code generation might automate some of this registration, or they might use macros to avoid writing boilerplate for each op.)
- **Autograd and Autocast:** Along with base kernels, TTNN likely registers Autograd formulas or lets autograd fall back. PyTorch's system has `AutogradPrivateUse1` keys for backward functions ³⁹. It's possible TTNN provides some custom backward implementations for performance (especially for things like linear layers or convolution, where they might call optimized backward kernels on the device). Otherwise, autograd will default to either using the functional fallback or a finite-difference if available. For automatic mixed precision (AMP), if relevant, PyTorch also allows registration of autocast policies (`AutocastPrivateUse1`). TTNN may or may not integrate deeply with AMP; since Tenstorrent primarily uses lower precision (BF16/8), the need for autocast might be minimal (the model is likely run entirely in BF16).

In Python, the `torch_ttnn` package exposes user-facing functions and classes. For example, `torch_ttnn.TorchTtnnOption` is a Python class that holds compilation options (device mesh info, parallelism, etc.) to pass into the backend ³⁴. There is also `torch_ttnn.backend` which is the entry-point for the compile backend (likely a Python function that TorchDynamo calls with the FX graph and options). This function probably delegates to a C++ or Python module in `tracer/` or similar to perform the lowering to TTNN graph. It might construct TTNN operations to mirror the FX graph, or invoke TTNN's own graph-capture APIs. Notably, TTNN has a C++ graph capture interface as shown in the docs (with `ttnn.graph.begin_graph_capture()` and `...end_graph_capture()` to get a graph object) ²⁷. It's conceivable that the compile backend uses a C++ graph capture under the hood: it could start graph capture, execute the model (or the FX graph) in a special no-dispatch mode, and obtain a captured TTNN graph. This TTNN graph could then be optimized or directly executed. The code snippet in docs captures a simple sequence of ops into a graph and even visualizes it ²⁷ ⁴⁰, suggesting that TTNN can represent graphs of ops internally. For the compiled backend, rather than executing ops one by one, it might build such a graph and then run it as a whole.

Summary of Execution Pipeline: Both eager and compiled modes ultimately call into the **TTNN C++ library**, which is built on TT-Metal, to handle device operations. The difference is scope: eager mode issues

one operation at a time to hardware, while compiled mode plans a whole sequence of operations. The **end-to-end flow** when running a model on Tenstorrent can be summarized as:

1. **Initialization:** Use `ttnn.open_device` (or `open_mesh_device`) to initialize the hardware and get a handle. Register this as a torch device (`ttnn_device_as_torch_device`) so PyTorch knows about it ⁴¹.
2. **Data Transfer to Device:** Move model parameters and input tensors to the TT device (e.g., `model.to(tt_device)` or using `ttnn.from_torch` for data ⁴²). This invokes TTNN's device allocator and tiling conversions.
3. **Execution (Eager):** If using eager mode, each `forward` call on the model will trigger numerous individual kernel launches as described, one per operation.
4. **Execution (Compiled):** If using compile mode, the first `forward` triggers graph capture; TTNN backend compiles the graph. Then a highly optimized single launch (or a small number of launches) will run the entire model on device. Subsequent forwards reuse the compiled graph.
5. **Transfers Back:** If the output is needed on CPU (or a non-TT device), call `tensor.to("cpu")` or `ttnn.to_torch` to transfer it back ⁴³ ¹⁹. Otherwise, one can keep the result on device for further processing to avoid overhead.
6. **Teardown:** After computation, `ttnn.close_device(device)` should be called to release the device context ⁴⁴. This is especially important if you are managing devices manually in long-running processes (to avoid resource leaks). In many simple scripts it may not be strictly necessary, but it's good practice.

Developing and Adding Eager Mode Functionality

Now that we understand the architecture, let's focus on **adding functionality in eager mode**, which is likely your task as a C++/Python developer joining the project. "Adding eager mode support" could mean a few things: implementing new operators to run on the Tenstorrent device (so they no longer fallback to torch), adding support for new data types or layouts, or extending device features (like support for new hardware generations or multi-device features in eager context). Most commonly, it means **adding a new TTNN operation** and hooking it into PyTorch. Here's how you generally approach this:

1. Implement the Operation in TTNN (C++): If the operation is not yet in TTNN's library, you start by adding it there. TTNN classifies ops as *device operations* (implemented directly via TT-Metal kernels) or *composite operations* (implemented by combining other TTNN ops) ⁴⁵. For example, if you want to support a new activation function, say `silu` (Sigmoid Linear Unit), you have choices: write a device kernel for it (maybe combining a sigmoid and multiply in one program) or simply compose existing ops (`silu(x) = x * sigmoid(x)`) as a composite. The TTNN docs provide a detailed guide on adding a new operation, with an example of adding a toy op called `ttnn.example` ⁴⁶. Key steps include: creating a struct for the operation, writing the device code (or the algorithm in C++), and registering it with TTNN's internal registry via `ttnn::register_operation` ⁴⁵ ⁴⁷. You'll also add Python bindings so it becomes accessible as `ttnn.some_op` in Python, using `ttnn::bind_registered_operation` (TTNN's mechanism to expose C++ ops to Python) ⁴⁸. This internal registration makes the op available within TTNN's own API (so one could call it directly on TTNN tensors).

2. Register PyTorch Dispatcher Kernel: After the TTNN operation exists, you integrate it into PyTorch's dispatcher so that `torch` operations invoke it. This means adding an entry in the `TORCH_LIBRARY_IMPL(aten, PrivateUse1, m)` section of the C++ extension for the corresponding

Aten operator. For example, PyTorch's Silu op is available as `aten::silu` (and as `torch.nn.functional.silu` in Python). You would implement a function, say `Tensor ttnn_silu_forward(const Tensor& input)` in the extension. This function will typically: (a) accept a `const at::Tensor&` (which is actually a wrapper around TTNN tensor if input is on TT device), (b) extract the underlying TTNN tensor (somewhere TTNN likely has a method to get a pointer/reference to its internal tensor from an `at::Tensor` – possibly stored in `TensorImpl`'s custom data), (c) call your TTNN C++ op, e.g. `auto result_ttnn = ttnn::silu(ttnn_tensor)`, (d) wrap the result back into an `at::Tensor` to return. Once this function is written, you register it:

```
TORCH_LIBRARY_IMPL(aten, PrivateUse1, m) {
    m.impl("silu", TORCH_FN(ttnn_silu_forward));
}
```

(The exact syntax may differ, but this is the idea.) After this, any time `silu` is called on a TTNN device tensor, your implementation will run. If you forget this step, the op would fall back to CPU by the generic fallback, which we want to avoid for performance. You will also consider if the operation has an in-place variant or out-of-place; those might be separate registrations (e.g., `"silu_"` for inplace). Many ops also have an Autograd part – if the backward is not automatically covered, you might register an Autograd kernel or rely on the autograd engine.

3. Fallback and Verification: Initially, you might implement the forward using simpler means and test it. TTNN encourages having a **golden reference (fallback)** for each op for testing ⁴⁹ ⁴⁸. In Python, you can attach a golden function to the op via `ttnn.attach_golden_function`. For instance, for `silu`, the golden function could be a lambda that takes a TTNN tensor, converts to torch, does `torch.nn.functional.silu`, and returns the result. This helps in testing correctness: TTNN has testing utilities that can run an op and compare to the golden output (possibly even for every op if you enable a debug mode that checks device vs fallback results). During development, you might run the test suite with an environment flag that uses the fallback (as in a “precision comparison” mode) ⁵⁰. The TTNN contribution guide explicitly says that when adding a new op, you should *first add a fallback implementation (reference) and tests*, and only then add the optimized implementation ⁵¹ ⁴⁹. This ensures that even before the device code is ready, the op behaves correctly (just slower via fallback). It's a test-driven approach: get the op working functionally, then optimize.

4. Implement Backward (if needed): If your op will be used in training (or even in inference models that call `torch.autograd.grad`), you should consider the backward. You can rely on PyTorch to do automatic differentiation by composing existing gradients (especially if your op is composite, autograd can often handle it). But for performance, you may implement a backward kernel too. This can be done by registering an Autograd kernel for the PrivateUse1 key. For example, if `silu` backward formula is `grad_input = grad_output * silu_backward(input)`, you might implement that similarly and register under `AutogradPrivateUse1`. The specifics can be complex; initially you might skip custom backward and let PyTorch fallback to CPU grad. However, note that if during backward it falls back, it will incur copies. TTNN's own docs and issues might guide which ops need custom backward urgently. Focus on forward first, but be aware of backward if targeting training use-cases.

5. Testing on CPU vs TT Device: After implementing the forward (and possibly backward), run TTNN's test suite. Write new tests if appropriate (e.g., to cover your op's functionality). Without hardware, you can set up

tests to use the fallback mode as validation. With hardware (or when you get access), test that the op produces correct results on-device and measure its performance. TTNN likely has a “sweep test” that runs a variety of ops with random inputs; ensure yours passes those. Also consider edge cases: broadcasting, different tensor shapes (including non-contiguous if TTNN supports that via tiled strides), dtype variants (float32 vs bfloat16 vs maybe int). Many elementwise ops would primarily be BF16 on Tenstorrent, but torch might call them with float32 on CPU then convert – TTNN could either support float32 (executing in BF16 under the hood, since hardware is BF16) or directly require BF16 inputs. The docs hint that `ttnn.from_torch` with dtype argument is used to cast to BF16 ¹⁹, so probably TTNN does expect certain dtypes. Make sure to handle dtype conversions appropriately in your op implementation.

6. Integrate with Compiled Mode: The last step is ensuring the compiled backend recognizes your new op. If you simply mapped an existing Aten op to TTNN, and the compiled backend was handling that op via fallback before, it might automatically use your new kernel now. However, sometimes you need to update the backend’s lowering logic. For instance, if the FX graph includes an op it previously didn’t know how to lower to TTNN, you might add a rule for it. This could be in Python (perhaps in `tracer/` code) where they have a dictionary or if-elif chain converting FX nodes to TTNN calls. Or TTNN might generate its graph by actually running operators (so if your eager op is implemented, the compile path might just call it during capture). Investigate TTNN’s backend implementation to see if new ops need manual registration there. A clue: TTNN’s docs have a section “Lowering TT-NN Operation to PyTorch” (likely about ensuring each TTNN op has a corresponding torch operator, perhaps for tracing). Ensure that the mapping from torch -> TTNN is complete for your op in both eager and graph modes.

Example – Adding an Operation (Toy Example): For a concrete illustration, suppose we want to add support for a new matrix operation, say a matrix elementwise exponential `aten::exp` on TT device (if it wasn’t supported already). The steps would be:

- Implement `ExpDeviceOperation` in TTNN C++ if not present (though likely TTNN has exp). This would involve writing a TT Metal kernel for exponential or using existing library support. In many cases, TTNN’s library might already have common ops like exp, so you might skip this if it exists.
- In `torch_tttnn` C++ extension, write `tttnn_exp(const at::Tensor& input)` that calls `tttnn::exp(input_tt)` and returns the result.
- Register it: `m.impl("exp", &tttnn_exp)` under `PrivateUse1`.
- Write a quick test: take a range of values, compute exp on CPU and on TT device (fallback or actual) and compare relative error.
- Run tests to ensure it doesn’t regress anything.

If Tenstorrent hardware supports exp natively (likely via an approximation in hardware), your kernel would utilize that. If not, you might implement exp as a series of simpler ops (e.g., a polynomial approximation) in TTNN – but that’s an edge case.

Documentation and Collaboration: When adding new functionality, update the documentation accordingly. This might include adding the op to the “Supported Python Operators” list in docs ⁵², adding usage examples if relevant, and noting any limitations (e.g., “operates in BF16 precision”). Internally, follow the contribution process – open an issue as described (they ask external contributors to file an issue labeled “tttnn” with the proposal) ⁵³, then after implementation, perhaps issue a PR referencing it. Tenstorrent has a bounty program for certain contributions, so they encourage communication via issues and discussions.

Remember to adhere to TTNN's coding standards and testing requirements (they use Apache 2.0 license ⁵⁴, and have formatting/lint configs in the repo). For C++ development, you'll be interacting with both PyTorch's API (ATen, C10 Dispatcher, etc.) and Tenstorrent's TT-Metal/TTNN APIs. It's a cross-section of two worlds, so expect a learning curve in both. Utilize the **DeepWiki** and developer docs provided (the TTNN deepwiki and TT-Metal deepwiki contain design notes and reference documentation). For example, TTNN's "Native Device Integration Extension" document (in the repo docs) likely details how they set up the PrivateUse1 device, which is valuable to read when modifying device-level functionality. Also, the "Operations Report" and "Known Issues" in docs may list current unsupported ops or bugs ⁵⁵ – this can guide you on what to tackle.

Conclusion and Whole Picture

By now, you should have a clearer picture of how PyTorch 2.0 TTNN integrates Tenstorrent hardware into the PyTorch ecosystem. In **summary**: PyTorch TTNN adds a custom device (using PyTorch's open backend API) so that tensors can reside on Tenstorrent accelerators. In **eager mode**, each PyTorch op is intercepted and executed through TTNN's C++ library on the hardware, maintaining correctness via fallback for any missing pieces. In **compiled mode**, PyTorch's new compiler technology is used to optimize and bundle operations, yielding massive performance gains by fully utilizing the Tenstorrent architecture. The architecture involves multiple layers – from Python integration, C++ extension, down to TT-Metal driver calls – but these are organized in a way that is largely transparent to the end user, who interacts with a familiar `torch` interface (just with an extra `ttnn` device or compile backend).

For a new developer on the project, the key areas to understand and contribute to are the **C++ extension (dispatch and kernels)** and the **TTNN library ops**. You'll be writing code that straddles PyTorch's internals and Tenstorrent's SDK. It's crucial to test thoroughly: use PyTorch's CPU as a reference to verify your TTNN outputs (the project provides tooling for that, like golden functions and debug modes). Keep an eye on how data is laid out (tiled vs contiguous) and when transfers happen – performance bugs often come from inadvertent host transfers or misaligned data. Also, be mindful of concurrency and device management; e.g., ensure your code works with multiple devices or multi-threaded dataloading (PyTorch might call your kernels from multiple threads), and use synchronization as needed (TTNN may require explicit `ttnn.synchronize_device()` calls at times ⁵⁶, e.g., after asynchronous launches, to match PyTorch's expected synchronous semantics in eager mode).

Finally, remember that TTNN is an active project (with an open issue tracker and Discord community ⁵⁷). Don't hesitate to engage with the maintainers – often they can provide guidance on the expected approach for adding a particular op or point out if the hardware has nuances (e.g., "operation X is not supported on Wormhole" or "needs tiling in blocks of 64" etc.). By following the project's contributing guide and the outline above, you'll help extend TTNN's eager mode capabilities, bringing more models to run **end-to-end on device** ²¹ and making the PyTorch-on-Tenstorrent experience ever more seamless.

Sources:

- Tenstorrent TTNN GitHub README and documentation ² ³ ²¹ ²³ for usage examples, fallback approaches, and model support status.
- PyTorch Developer Tutorials on custom backend integration ¹⁴ ¹⁵ (illustrating the use of PrivateUse1 dispatch keys and kernel registration) which align with TTNN's integration strategy.
- Martin's Tenstorrent blog for hardware context and TTNN's design on tiling and TT-Metal ¹² ¹¹.

- Official TTNN documentation for API usage and contributing process ¹⁸ ⁵¹ , which guided the steps outlined for adding new operations.

¹ ² ³ ⁴ ¹³ ²¹ ²² ²⁸ ²⁹ ³⁰ ³¹ ³⁴ ³⁵ ³⁶ ⁴¹ ⁵⁴ ⁵⁵ ⁵⁷ GitHub - tenstorrent/pytorch2.0_ttnn: ☆ TTNN Compiler for PyTorch 2 ☆ Enables running PyTorch models on Tenstorrent hardware using eager or compile path

https://github.com/tenstorrent/pytorch2.0_ttnn

⁵ ⁶ ⁷ ⁸ ⁹ ¹⁰ ¹¹ ¹² ¹⁶ ¹⁷ ³² ³³ Building new GGML backends for novel accelerators, how, challenge and opportunities (FOSDEM 2025 draft) - Martin's website/blog thingy
<https://clehaxze.tw/gemlog/2024/12-28-building-new-ggml-backends-for-novel-accelerators-how-challenge-and-opportunities-fosdem-2025-draft.gmi>

¹⁴ ¹⁵ ²⁰ ³⁷ ³⁹ Facilitating New Backend Integration by PrivateUse1 — PyTorch Tutorials 2.8.0+cu128 documentation

<https://docs.pytorch.org/tutorials/advanced/privateuseone.html>

¹⁸ ¹⁹ ²³ ²⁴ ²⁶ ²⁷ ⁴⁰ ⁴² ⁴³ ⁴⁴ Using TT-NN — TT-NN documentation

<https://docs.tenstorrent.com/tt-metal/latest/ttnn/ttnn/usage.html>

²⁵ ³⁸ Any simple example about the new way to register custom device through PrivateUse1 - frontend API - PyTorch Developer Mailing List

<https://dev-discuss.pytorch.org/t/any-simple-example-about-the-new-way-to-register-custom-device-through-privateuse1/1561>

⁴⁵ ⁴⁶ ⁴⁷ ⁴⁸ Adding New TT-NN Operation — TT-NN documentation

https://docs.tenstorrent.com/tt-metal/latest/ttnn/ttnn/adding_new_ttnn_operation.html

⁴⁹ ⁵⁰ ⁵¹ ⁵³ Onboarding New Functionality — TT-NN documentation

<https://docs.tenstorrent.com/tt-metal/latest/ttnn/ttnn/onboarding.html>

⁵² ⁵⁶ Welcome to TT-NN documentation! — TT-NN documentation

<https://docs.tenstorrent.com/tt-metal/latest/ttnn/index.html>